

PSE: Einleitung Implementierung



Lukas Hilgert

Lehrstuhl KI für Sprachtechnologien (AI4LT)

Übersicht

- Artefakte
- Plan
- Bericht
- Unit-Tests
- Sonstiges
- Inhalt der Präsentation

Inhalt von

https://sdq.kastel.kit.edu/wiki/Praxis_der_Software-Entwicklung/Tipps_und_Tricks

Artefakte

- Artefakte
 - Implementierungsplan (zu Beginn)
 - Implementierungsbericht (PDF-Dokument)
 - Vollständiger Quellcode
- Umfang
 - 20 Seiten (Bericht)

Plan

- Klarer zeitlicher Ablauf der Implementierungsphase, um frühzeitig Verzögerungen zu bemerken.
- Abhängigkeiten aus dem Entwurf müssen beachtet werden.
- Wie können Teile der Anwendung möglichst früh getestet werden? Braucht es dafür Stub- / Mock-Klassen?
- Klare Aufgabenverteilung im Team. Dabei muss eine faire Verteilung und die Abhängigkeiten beachtet werden.
- Einzelne Jobs kann man Klassen oder Packages zuordnen, aber häufig ist das nicht möglich.
- Als Form bietet sich ein GANTT Chart an.
- Zu *Beginn* der Implementierungsphase (d.h. nach 1-2 Tagen) beim Betreuer abzugeben.

Plan

Bericht

- Erfahrungsgemäßer Umfang: ca. 20 Seiten
- Einleitung mit Anschluss auf Pflichtenheft und Entwurf
- Dokumentation über Änderungen am Entwurf,
 - beispielsweise entfernte oder neu hinzugefügte Klassen und Methoden.
 - Gruppiert (und zusammengefasst) werden sollte nach dem Grund für die Änderung und nicht nach der geänderten Klasse.
- Welche Muss- und Wunschkriterien sind implementiert?
- Welche Verzögerungen gab es im Implementierungsplan?
 - Kann beispielsweise als zweites GANTT Diagramm am Ende dargestellt werden.
- Übersicht zu Unittests

Unit-Tests

- Von Anfang an Unit-Tests benutzen. Diese Tests gehören *nicht* in die Phase Qualitätssicherung.
- Vor allem nicht-graphische Klassen können so frühzeitig getestet und Regressionen vermieden werden.
- Nur öffentliche Schnittstellen testen. Private Methoden werden nicht getestet.
- Unit-Tests testen nur kleine *units* (üblicherweise einzelne Klassen) für sich. Es sollen nicht ganze Testszenarien getestet werden.

Sonstiges

- **Warnungen** reparieren.
 - Es ist empfehlenswert, dass ein Projekt beim Bauen keine Warnungen ausgibt.
 - Eine Warnung bedeutet, dass der Code üblicherweise aber nicht garantiert fehlerhaft ist.
 - In Ausnahmen kann der Programmierer in Java dann Annotationen einfügen, um Warnungen zu unterdrücken.
- Warnungen anschalten.
 - In Eclipse kann man zusätzliche Warnungen anschalten: Project properties → Java → Compiler → Errors/Warnings.
 - Standardmäßig wird das meiste ignoriert.
 - Beispielsweise kann man aktivieren, eine Warnung anzuzeigen wenn eine Klasse equals überlädt, aber nicht auch hashCode.

Sonstiges

- Kommentare im Code (also nicht API Doku) sollten das *Warum* klären, nicht das *Wie*.
 - Es ist naturgemäß schwierig vorherzusehen, welche Warum-Fragen sich jemand stellt, der ein Stück Code liest.
 - Die Fragen sind üblicherweise »Warum ist X hier notwendig?«, »Warum ist kein X hier?« oder »Warum X, wobei Y doch besser wäre?«
- *Exceptions* (nicht *RuntimeException*) für Fehler beim Aufrufer.
 - Assertions für interne Konsistenzfehler bzw. um Annahmen explizit zu machen.
- Keine Magic Numbers
 - Explizite Zahlen im Quellcode sind entweder selbsterklärend oder durch benannte Konstanten zu ersetzen. Am besten immer letzteres.

Inhalt der Präsentation

- Was wurde implementiert? Welche Wunschkriterien gestrichen?
- Zeitplan eingehalten? Wo nicht?
- Unerwartete Probleme bei der Implementierung?
- Größere Änderungen am Entwurf?
- Grobe Statistiken:
 - Lines of Code
 - Wie viele Commits
 - Arbeitsaufteilung im Team
- Entwicklungsmodell
 - Wie wurde im Team kommuniziert?
 - Wie wurde Git benutzt:
 - a. Gab es Feature-Branches?
 - b. War das Mergen in den master/main-Branch beschränkt?
 - c. ...